UNIVERSITÄT BERN

Applied Optimization
Exercise 0 - Programming environment setup & Theory background

September 18, 2025

What to hand in

A .zip compressed file renamed to `Exercise`*n*`-Group.zip` where *n* is the number of the current exercise sheet. It should contain:

- Hand in **only** the files you changed (headers and source). It is up to you to make sure that all files that you have changed are in the zip.
- A `readme.txt` file containing a description on how you solved each exercise (use the same numbers and titles) and the encountered problems. Indicate what fraction of the total workload each project member contributed.
  Failure to do so will result in you losing a point.
- Other files that are required by your `readme.txt` file. For example, if you mention some screenshot images in `readme.txt`, these images need to be submitted too.
- For the theory exercise, put `TheoryExercise.pdf` with your solutions in the same .zip you submit for the code.
- Submit your solutions to ILIAS before the submission deadline. Late submissions will receive 0 points!
- **Important:** Your submission MUST compile without any issue in order to obtain a passing grade.

Introduction to unit testing

Throughout this course, you will have to fill in specific functions within our coding framework. Aside from the main exercises, which will often ask you to use a command-line tool to perform some operations, we will provide *unit tests* to test your implementations. Those tests will initially fail but should pass once your solutions are correct. Each exercise will thus contain two executables: the main one and the unit testing one. e.g., for this exercise, you will have both the `GridSearch` executable, showing the behavior described below, and the `GridSearch-test` executable running the unit tests. However, note that passing the unit tests does not guarantee a perfect grade. Those tests are only meant to help you partially validate your implementation. You will find more information on unit testing on Wikipedia.

Implementing objective functions (1 pt)

First, you need to download `aopt-exercise0.zip` and extract it into a folder of your choice. Follow the instructions in the exercise slides and compile the project.
In optimization terminology, we denote the function we want to minimize or maximize an objective function. Below are several functions that you need to implement in the code.

Quadratic function: $f(x) = \frac{1}{2}x^T Ax + bx + c$, with $A \in \mathbb{R}^{n\times n}$ and $b, x \in \mathbb{R}^n$.

2D Let $n = 2$, such that our variable is now a 2D vector in $x = (x_1, x_2)^T$. In this case, implement the 2D quadratic function:

$$f(x) = \frac{1}{2}(x_1^2 + \gamma x_2^2)$$

in the `eval_f(...)` function in `FunctionQuadratic2D.hh`. By default, γ is set to -1 in the code. Evaluate it on a reasonable domain and generate a .csv file to plot. The tool to export the .csv file is provided. Once the project is compiled, one can find a command line tool called "CsvExporter" in the build folder. Run it to see the usage. Afterwards, one needs to visualize the data on the website and submit a screenshot of it.

ND In applied optimization, the problem is often in higher dimensions. Implement the n-dimension quadratic function given above in the `eval_f(...)` function in `FunctionQuadraticND.hh`. The matrix A and vector b, c are initialized that one can use directly. For this function, there is no need to export data and visualize it.

A non-convex function: Consider the following expression with variables x, y in $\mathbb{R}$.

$$h(x,y) = (y - x^2)^2 + cos^2(4 * y) * (1 - x)^2 + x^2 + y^2$$

Implement it and evaluate it for visualization. Figure 1 depicts a visualization for comparison in the ranges $x \in [-2,2]$, $y \in [-2,2]$ on a grid of 100×100.

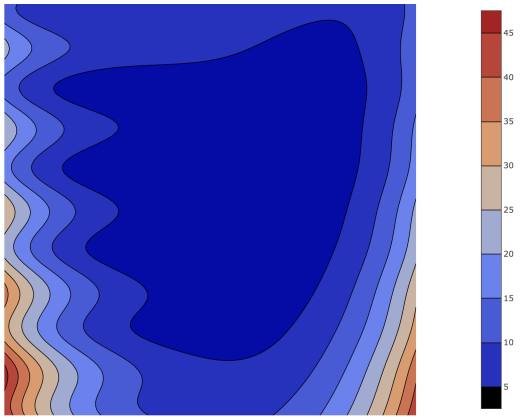

Figure 1: non convex function visualized as height map with isocontours

Grid Search (1 pt)

Grid search refers to an exhaustive search for an optimum (minimum or maximum value) function value. One can sample the variables as a grid (here, we use a uniform grid for simplicity) over a certain domain and evaluate the function values of all the grid points. Compare the function values and return the optimum in the end. One needs to implement grid search algorithms for the functions above that output the minimum function value among the grid points and the corresponding variable value. Specifically, the functions `grid_search_2d(...)` and `grid_search_nd(...)` need to be implemented in the `GridSearch.hh` file. The `grid_search_nd(...)` should work for any specified n input from the command line. Note that n represents the number of *cells* on one side and not the number of *points* on one side. Run the command line tool "grid_search" to see the usage. Experiment with different functions and different parameters. See how inefficient the method is for high-dimension optimization.

Note: You can solve this exercise with an iterative or recursive approach. Feel free to add any necessary function to make your code more readable/better structured.

Eigen Tutorial (optional)

Aside from the mandatory exercises above, you can get acquainted with the `Eigen` library by doing the very short tutorial provided alongside the main exercises. First, download the `aopt-eigen-tutorial` from ILIAS, extract it and build it the same way you built `exercise0`. You can now follow the instructions in the `unit_tests.cc` file and solve each unit test in there. Your solution can be checked by running the `EigenTutorial-test` executable.

Theory Background Exercises (optional)

Those optional exercises should give you an idea of the theoretical background necessary to follow this course. You should be able to do them all without much hassle. If they prove to be complicated for you, we **very strongly** encourage you to catch up quickly and practice all topics tested here. Note that you can submit your solutions if you want us to grade them.

Functions and Domain Sketches

Sketch the following functions, curves, and domains:

1. $f(x) = x^3, x \in [-1,1]$
2. $\{x | x < 1\}$
3. $\{(x,y) | x^2 + y^2 = 9\}$
4. $\{(x,y) | y = x - 2 \wedge x = 6 - y\}$

Multivariate Functions and their Derivatives

For each of the following functions $f_i : \mathbb{R}^{n_i} \to \mathbb{R}$, compute their gradient ∇f_i and Hessian matrix H_f:

1. $f(x,y) = x^2 + y^2$
2. $f(x,y,z) = 2xy + x^2 + y^2 + z(x^2 - y^2)$
3. $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{x}$, with $\mathbf{x} \in \mathbb{R}^n, n > 2$

For each of the following functions $f_i : \mathbb{R}^{n_i} \to \mathbb{R}^{m_i}$, compute their Jacobian matrix J_i:

1. $f(x,y,z) = (2xy + x^2 + y^2, \quad z(x^2 - y^2))$
2. $f(x,y) = \begin{pmatrix} x^2 y \\ x - y \\ 3xy^2 \end{pmatrix}$
3. $f(\mathbf{x}) = \mathbf{x} \odot \mathbf{x}$, where $\mathbf{x} \in \mathbb{R}^n, n > 2$ and $\odot$ represents the element-wise multiplication.

Linear Systems

Solve the following linear systems:

1. $\begin{cases} x + y + z = 2 \\ x - y - z = 10 \\ x + 2y - 3z = 5 \end{cases}$
2. $\begin{cases} x_1 + 2x_3 - x_5 = 0 \\ 2x_2 + 2x_3 - 2x_5 = 1 \\ x_3 + 2x_4 - x_5 = 2 \\ 2x_2 + 2x_3 - x_5 = -2 \\ x_4 - 2x_5 = -1 \end{cases}$

Engineering example

Suppose we use the following model with unknown parameters to describe a specific chemical process.

$$y(x) = \beta_0 e^x + \beta_1 sin(\tfrac{2\pi}{3}x) + \beta_2 x^3,$$

where β_i - are the parameters. To perform data fitting and roughly estimate the parameters of the model, we use linear regression over a small set of inputs $\{x_i\}_{i=1}^4$, and corresponding outputs $\{y_i\}_{i=1}^4$.

i	x_i	y_i
1	-1	-5
2	0	1
3	1	6
4	2	34

In the following, you are tasked with performing some of the necessary steps to solve the problem.

1. Rewrite the model equation in a format $y(x) = \mathbf{a}(x) * \mathbf{X}$, where $\mathbf{X} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$ is a column vector of unknown parameters, and $\mathbf{a}(x)$ - is a row vector of corresponding functions.

2. Using the rewritten model equation and the provided data, formulate an overdetermined linear system $\mathbf{Y} = \mathbf{A} * \mathbf{X}$, where $\mathbf{Y} = \begin{bmatrix} .. \\ y_i \\ .. \end{bmatrix}$ is a vector of outputs of the model, and $\mathbf{A} = \begin{bmatrix} ..... \\ \text{—— } \mathbf{a}(x_i) \text{ ——} \\ ..... \end{bmatrix}$ is a model matrix, formed of the above-mentioned row vectors, evaluated for each given input. Note that since the matrix represents the model evaluated for the fixed dataset, the x dependence is omitted. $\mathbf{X}$ is the above-mentioned vector of parameters, and $*$ represents a standard matrix-vector product.

3. Since the problem is overdetermined; a proposed solution technique is to optimally satisfy the data in the 2-norm sense. Algebraically put, the problem is as follows.

   $$min||\mathbf{Y} - \mathbf{A} * \mathbf{X}||_2^2$$

   Expanding the definition of the 2-norm, show that this formulation minimizes the quadratic objective function.

4. Compute the gradient of the resulting function.

5. Compute the Hessian of the corresponding function.

6. Is the Hessian symmetric positive-definite? Check by computing the eigenvalue decomposition of the Hessian. (*Hint.* For symmetric matrices, positive (negative) definiteness means that all of the eigenvalues are strictly positive (negative). If, however, at least one of the eigenvalues is zero, but the rest have the same sign, the matrix is called positive (negative) semi-definite. If none of the above holds, the matrix is called indefinite.)